

Adaptive HACT: Semantics-Preserving Workload-Adaptive Verification Evidence for Agentic Software Systems

Research manuscript*

October 6, 2026 | Journal manuscript v6.0 | Artifact 5.0 + frozen journal extension

Abstract

Agentic software pipelines increasingly gate candidate changes on tests, analyses, and policy checks, yet the evidence supporting a commit decision can be much larger than the final verdict. Adaptive HACT studies whether hierarchical verification evidence can be reorganized for lower diagnostic-delivery cost without changing what is accepted. Its central abstraction is *evidence-layout independence*: immutable check identities and candidate/checker/environment bindings define semantics, while a separate layout plane only aggregates trusted completion records. Generation fencing makes authorization invariant to valid layout permutations and migrations. Because refresh cost depends on which *sets* of checks complete together, candidate layouts are scored on completion hyperedges rather than on marginal or pairwise statistics alone. The practical compiler learns an average-linkage order and uses a compact balanced eight-ary tree, retaining the incumbent through validation.

A precommitted, disjoint-seed replication evaluates 180 train/validation/held-out workloads (20 seeds for each of three sizes and three completion families). On clustered completions, validated HACT reduces held-out padded evidence by 23.56% (95% bootstrap CI 23.20–23.94), 10.66% (10.42–10.91), and 5.26% (5.16–5.37) at 128, 512, and 1,024 checks; all 60 clustered replications improve, with one-sided seed-level sign-test $p = 9.54 \times 10^{-7}$ at each size. The corresponding marginal-frequency baselines reach 15.63%, 4.90%, and 2.01%, while validated random orders remain near zero. Independent controls have confidence intervals spanning zero and global controls are exactly invariant. In a separate 60-workload distribution-shift study, stale layouts deliver essentially no mean benefit, whereas post-shift retuning restores 23.58%, 10.99%, and 5.41% reductions with no observed regressions. A previously frozen 33-case real-source confirmation cohort yields a 20.30% mean reduction for the exact conditional compiler versus fixed8 (28/33 positive, sign-test $p = 3.31 \times 10^{-5}$), while twelve codec replays of the practical balanced layout reduce whole-epoch compressed bytes by 4.36–41.84%. Across replayed layouts, semantic verdicts agree and root-status text remains byte-identical. The result is a semantics-preserving, workload-adaptive evidence representation with quantified benefit, failure modes, and deployment boundaries—not a universal agent, token, or WAN accelerator.

Keywords: agentic systems; verification evidence; completion hyperedges; evidence-layout independence; bounded diagnostics; cost-aware layout selection.

1 Introduction

Autonomous and multi-agent software systems can generate many candidate actions in parallel, but accepting a candidate is a different problem from generating it. A deployment gate may need evidence from tests, static analyzers, policy checks, environment probes, and integration checks. The final decision may be one bit, yet debugging, audit, and remote supervision can require a much larger transcript of intermediate evidence. That creates a representation problem: reorganizing the transcript may save bytes and retrieval work, but it must never change which checks are required or turn missing evidence into a pass.

Adaptive HACT treats verification semantics and evidence representation as separate planes. The semantic plane binds a candidate to a registry of stable check identities and to the checker and environment

*Author names and affiliations require confirmation before submission. Mathematical development, implementation, experiments, and writing were AI-assisted.

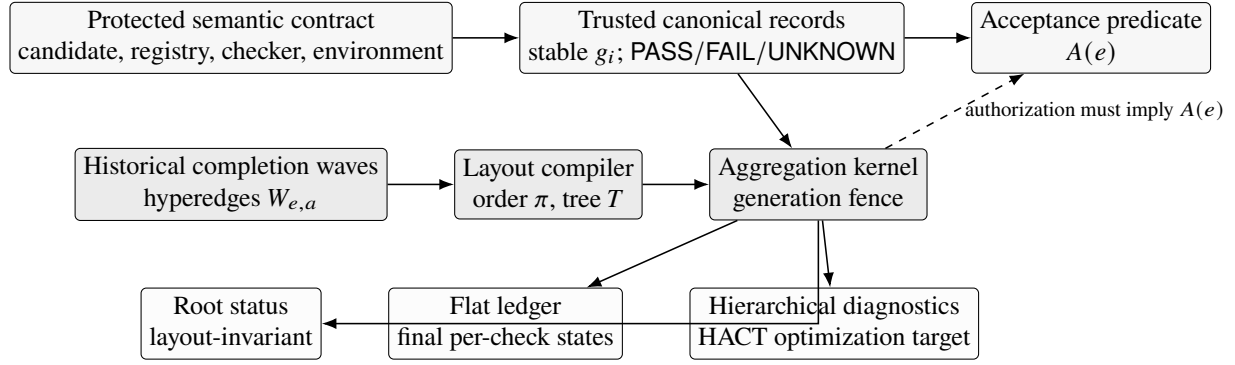


Figure 1: Adaptive HACT factors verification semantics from evidence representation. A valid layout may change aggregation cost and diagnostic hierarchy, but it cannot change the approved check set or the acceptance predicate. Root status and flat final ledgers are simpler services; HACT targets deployments that actually require hierarchical diagnostic evidence.

that produced each result. The layout plane is free to permute those identities and aggregate their records into bounded packets. This separation creates a useful systems invariant: layout learning can change *where evidence is stored and how much is emitted*, but not *what must be true before authorization*.

Evidence-layout independence. The paper formalizes the invariant sketched in Figure 1. A semantic registry and candidate identity are authoritative. Layouts are advisory data structures over those identities. A generation change invalidates layout-specific publication tickets and forces reconstruction from canonical trusted records. Under the stated trust assumptions, Theorem 5 shows that every authorized layout yields the same acceptance decision. This makes representation learning compatible with a fail-closed gate.

Why completion hyperedges matter. Evidence cost is driven by refresh events: when a set of checks becomes final, every ancestor packet intersecting that set may need to be re-emitted. Marginal activation rates lose joint structure, and pairwise co-activity can still miss the probability that a larger subtree is touched. We therefore treat each completion wave as a hyperedge and score candidate layouts on the observed sets directly. Pairwise affinity is used only to propose an order; it is not substituted for the actual full-wave objective.

Where HACT is useful. There are three distinct services. A consumer needing only a trusted final verdict should use a root-status projection. A consumer needing final per-check states can often use a flat ledger. HACT addresses the third case: a system that already needs bounded hierarchical evidence for diagnostics, audit, or constrained delivery. This service distinction prevents a common accounting error—crediting evidence compression with the unrelated cost of introducing a stricter verifier.

Contributions. The paper makes four claims, each tied to a concrete proof or experiment:

1. **A semantic/representation factorization.** We define evidence-layout independence and give a generation-fenced implementation whose local authorization is invariant to valid permutations and migrations (Section 3; Theorem 5).
2. **A hyperedge-sensitive cost model and compiler.** Full completion waves determine layout cost; Proposition 2 shows why pairwise affinity alone is insufficient. A practical no-DP compiler combines learned ordering with a balanced eight-ary tree, while exact alphabetic DP provides a fixed-order optimum for reference (Section 4).
3. **Migration- and service-aware operation.** Layout changes pay an explicit installation price, and deployment accounting compares codec-matched services instead of mixing bytes, tokens, and unrelated harness time (Sections 5 and 8).

4. **Precommitted replication, robustness, and source confirmation.** A frozen 180-workload disjoint-seed replication, 60 post-shift workloads, 33 held-out real-source confirmations, 36 codec/layout replays, invariance checks, and an independent recomputation auditor quantify structured gains, null regimes, drift sensitivity, and deployment tradeoffs (Section 9).

Clustering, balanced trees, alphabetic DP, compression, and break-even arithmetic are established ingredients. The contribution is their use under a verification-semantic invariant and a completion-hyperedge objective, together with evidence that distinguishes the useful regime from the null controls.

Scope. The experiments measure verification-evidence representation and local integration. They do not measure live-LLM coding quality, physical-WAN performance, or provider token billing. Those boundaries are collected in Section 10 instead of being repeatedly mixed into each positive result.

Roadmap. Section 3 defines semantics and the cost model. Section 4 develops the practical and exact-reference compilers; Section 5 prices migration; Section 6 proves layout-independent local publication soundness. Sections 7 and 8 describe the implementation and service-matched accounting. Section 9 answers six evaluation questions with precommitted replication, source confirmation, drift analysis, and deployment accounting; Section 11 states the deployment implication.

2 Related Work and Positioning

Testing, impact analysis, and incremental computation. Test prioritization seeks earlier fault exposure [1]; regression-test selection uses change dependencies [2, 3]. The earlier IC-HACT ranker uses controlled mutation outcomes to prioritize checks, an instance of established fault-exposure methods. It is held fixed in the new source comparisons. Self-adjusting computation and build-system research already formalize dependency-aware recomputation [4, 5]. In contrast to test selection, our strict pass requires the entire approved registry on the identified candidate. We do not reproduce NameRTS or Ekstazi and claim no superiority over them.

Seriation and authenticated layouts. Spectral seriation orders items using an affinity matrix [13]. Average-linkage clustering likewise provides an established proximity-based ordering. Generalized alphabetic-tree optimization supplies the subtree/forest recurrence used here [6]. Workload-adaptive authenticated structures, including Huffman–Merkle Trees [7], already optimize layouts under access patterns. Our price counts repeated completion-frontier updates, including strict-epoch construction, under a fixed packet cap. Pairwise affinity proposes an order; full hyperedge observations score it. No global optimality or unproved NP-hardness result is claimed for unrestricted joint ordering and tree synthesis.

Online decisions with switching costs. Fixed-share learning tracks changing experts [14]; dependent sampling reduces switches compared with independent weighted-majority draws [15]. Full-information and bandit switching-cost settings have different achievable regret guarantees [16]. We use the full-information setting because one recorded completion wave can be scored against every frozen certificate layout without reexecuting the tests. This is not full counterfactual feedback for arbitrary agent strategies: if a layout changes the tests’ execution schedule or model behavior, our exogenous-cost assumption may fail. Our bound specializes established tools to measured layout-installation costs; it is not a new minimax online-learning result.

Agent runtime assurance and evaluation. Proof-Carrying Agent Actions [8] and verification-portfolio commit gates [9] address related action-governance problems. Our local certificates are neither signatures nor zero-knowledge proofs. SWE-bench evaluates real repository issues and patches [17]; our source

mutations and scripted I/O repairs must not be called SWE-bench results. The current integration measures real orchestration and verification, not model reasoning quality.

Logging and serialization controls. Telemetry systems already batch and compress records [18]. HACT is not a replacement for those techniques: fewer padded bytes need not imply fewer compressed bytes. We measure both. Moreover, flat result ledgers and a trusted root-status endpoint can be much smaller than an archive of all intermediate aggregation updates; they satisfy different delivery needs and must not be hidden from the comparison.

3 Model and Trust Boundary

3.1 Semantic registry, execution, and display

Let $\mathcal{R} = (g_0, \dots, g_{n-1})$ be a nonempty approved registry of unique check IDs. The candidate identity is

$$e = (\text{epoch}, \text{snapshot}, \text{checker}, \text{environment}, \text{registry}).$$

The snapshot covers source and protected acceptance inputs. The checker and environment are explicitly bound; an epoch increases even after an ABA return to identical source bytes. Only a trusted runner submits final PASS, FAIL, or UNKNOWN records. Missing, stale, skipped, incompatible, or timed-out outcomes are not passes.

An execution permutation σ_e determines when checks run. A layout $L = (\pi, T)$ independently maps tree positions to semantic registry indices. For a node covering positional interval $[i, j]$, its semantic coverage is

$$G_L(i, j) = \{g_{\pi(i)}, \dots, g_{\pi(j)}\}. \quad (1)$$

The stored layout must contain an exact permutation, and every internal node's children must partition its positional interval without gaps or overlaps. The task graph is outside this mathematical tree; one worker can discharge many checks, and one integration check can cover several workers.

The acceptance predicate is independent of both permutations:

$$A(e) = \mathbf{1}\{\text{every } g_i \in \mathcal{R} \text{ has trusted, matching, current PASS evidence for } e\}. \quad (2)$$

An observed fail permits rejection before all checks finish. It does not assert coverage of the unexecuted remainder. Acceptance is relative to the approved predicates, not proof of all program behavior.

Definition 1 (Evidence-layout independence). Fix a candidate identity e , semantic registry $\mathcal{R}$, and canonical trusted record set E . A verifier is *evidence-layout independent* if valid layouts over the same registry may change representation cost, retrieval structure, and packet timing but cannot change the semantic projection of E , and no layout can authorize unless $A(e)$ holds.

Definition 1 is deliberately local to the declared verification contract. It does not claim that the contract itself is complete or that the checker is uncompromised; those are trust assumptions stated explicitly in Theorem 5.

3.2 Packet and lifecycle costs

Each internal node of arity d emits a fixed-width packet

$$c(d) = H + Sd, \quad 2 \leq d \leq b = \lfloor (C - H)/S \rfloor. \quad (3)$$

Here $H = 512$, $S = 320$, and $C = 3072$ bytes, so $b = 8$. A mandatory field that exceeds its slot is rejected, not silently truncated. The cap applies to one certificate packet, not to all project memory, an entire registry

manifest, or every LLM prompt. A manifest is a referenced control-plane artifact; its measured size is charged on migration.

Let $W_{e,1}, \dots, W_{e,q_e}$ be the semantic IDs whose final records become available between successive refreshes. For a strict new candidate epoch, the observed cost is

$$\ell_e(L) = W(L) + \sum_{a=1}^{q_e} \sum_{v \in \text{int}(T)} c(d_v) \mathbf{1}\{W_{e,a} \cap G_L(v) \neq \emptyset\}, \quad W(L) = \sum_v c(d_v). \quad (4)$$

The first term constructs the full initially unknown tree. The CLI batches an executed prefix into groups of eight, flushing its final partial group. A successful strict epoch runs the whole registry. This conservative implementation does not reuse a pass across distinct source snapshots.

We call each $W_{e,a}$ a *completion hyperedge*: an arbitrary set of semantic obligations that becomes final between two refreshes. Equation (4) consumes these sets directly through interval-hit events. Pairwise statistics may propose a permutation, but the scored objective remains higher-order.

We separately study persistent-state refresh traces that omit $W(L)$ at each tick, as in a cache with correctly specified incremental dependencies. Each topology installation is still cold and explicitly priced. These traces evaluate the controller’s cost model; they are not the strict candidate-gate timing experiment. They do not demonstrate safe dependency inference from arbitrary code.

3.3 Why more than invalidation or pairwise similarity is needed

The inherited HACT result shows that marginal check activation cannot identify the best aggregation tree under joint invalidation (Appendix C). IC-HACT further shows that even the full initial invalidation distribution cannot identify optimal progressive verification: early rejection and completion batching change the update sequence. Both distinctions remain relevant after allowing permutations.

Proposition 2 (Pairwise affinity is insufficient for full-wave scoring). *There exist two distributions with identical singleton and pairwise co-activity probabilities, hence identical Jaccard affinities, but different activation probabilities for the same three-check subtree.*

Proof. On IDs $\{0, 1, 2\}$, let P be uniform on $\emptyset, \{0, 1\}, \{0, 2\}, \{1, 2\}$ and Q uniform on $\{0\}, \{1\}, \{2\}, \{0, 1, 2\}$. Every singleton has probability $1/2$, every pair has joint probability $1/4$, and its Jaccard affinity is $1/3$. Yet the probability that the triple is hit is $3/4$ under P and 1 under Q . Thus a packet emitted on that interval has different expected cost. $\square$

This elementary parity construction is not a new fact about higher-order statistics. Its implication for the compiler is specific: a pairwise clustering objective cannot substitute for scoring recorded completion hyperedges. It does not, by itself, prove that these two distributions have different globally optimal layouts.

4 Learning Layouts Without Changing the Contract

4.1 Practical catalogue and full-wave selection

Training waves define counts N_i, N_{ij} and affinity

$$a_{ij} = \frac{N_{ij}}{N_i + N_j - N_{ij}}, \quad (5)$$

with zero for an unobserved union and zero diagonal. The practical proposal uses established average-linkage clustering: join the closest current clusters, choose endpoint orientation by cross-boundary affinity, and break ties canonically. All semantic IDs, including disconnected and unseen checks, remain included exactly once.

Given the learned order, group consecutive leaves into blocks of at most eight, promote a singleton unchanged, and repeat upward. This bottom-up compact balanced construction does not call a tree optimizer. The catalogue retains the incumbent first, includes original-order fixed8 when distinct, and proposes learned-order fixed8. Identical layouts are deduplicated. A separate validation cohort scores complete completion waves, including initial construction; ties retain the incumbent. The API returns an advisory choice, not deployment authorization. The registry and packet cap cannot change during selection.

For any proposal π , training episodes supply interval weights

$$r_\pi(i, j) = 1 + \frac{1}{m} \sum_{e=1}^m \sum_a \mathbf{1}\{W_{e,a} \cap G_L(i, j) \neq \emptyset\}. \quad (6)$$

These weights also define the exact-DP reference below. Pairwise affinity *proposes* an order; it does not replace the full-wave cost. The practical catalogue contains at most three candidates and uses no DP. Tree construction is $O(n)$ after ordering, not a linear-time claim for affinity construction, clustering or validation scoring. The reference implementation retains spectral ordering and exact fitting for research comparisons; neither is invoked by the practical default.

4.2 Exact conditional tree synthesis

Let $D(i, j)$ be the optimal subtree cost and $F_k(i, j)$ the optimal cost of an ordered forest with k nonempty children. With infeasible states equal to $+\infty$,

$$D(i, i) = 0, \quad (7)$$

$$F_1(i, j) = D(i, j), \quad (8)$$

$$F_k(i, j) = \min_{i+k-2 \leq u < j} \{F_{k-1}(i, u) + D(u+1, j)\}, \quad (9)$$

$$D(i, j) = \min_{2 \leq k \leq \min(b, j-i+1)} \{r_\pi(i, j)c(k) + F_k(i, j)\}. \quad (10)$$

Theorem 3 (Exactness conditional on order). *For a fixed π , nonnegative interval weights, and the packet model in Equation (3), Equation (10) returns a minimizing feasible tree in $O(bn^3)$ time and $O(bn^2)$ space.*

Proof. Every feasible root has an arity and a contiguous partition of its interval. Its local contribution depends only on that interval and arity; all remaining contributions belong to independent child subtrees. The forest recurrence enumerates every such partition. Induction on interval length proves exactness. There are $O(bn^2)$ states, each scanning at most n splits. The apparent $F_1 = D$ dependency is used only on shorter intervals when constructing a forest with at least two components. $\square$

This is a specialization of classical alphabetic-tree optimization [6]. The historical three-proposal reference multiplies compilation time by a constant, not by $n!$. Dense spectral computation can itself cost $O(n^3)$; the reference clustering implementation uses $O(n^2 \log n)$ heap operations and $O(n^2)$ storage.

Proposition 4 (Validation after learned ordering). *Condition on a training set that fixes K complete layouts. For m independent identically distributed validation epochs with each layout cost in $[0, B]$, let $\widehat{L}$ minimize empirical validation cost. With probability at least $1 - \delta$,*

$$J(\widehat{L}) \leq \min_{L \in C} J(L) + 2B \sqrt{\frac{\log(2K/\delta)}{2m}}. \quad (11)$$

Proof. Conditionally, the catalogue is fixed. Hoeffding's inequality [10] and a union bound over K layouts bound every empirical mean error by the displayed radius without its factor two. Inserting empirical costs on either side of the empirical minimizer yields the result. $\square$

The guarantee is only relative to the fixed catalogue. It neither justifies choosing among learned permutations on their training errors alone nor covers distribution shift. We do not report this potentially loose bound as an empirical confidence interval; reported intervals use independent experimental seeds.

4.3 Compiler identity and scope

The practical default has no hidden microtree optimization. Exact DP is a reference and optional offline candidate. The block-backbone proposition and implementation remain in Appendix B solely as an ablation; its restricted-class exactness does not override the observed simpler-baseline advantage. At 1,024 checks, order learning still takes seconds in our measurement, so layouts should be cached and compiled away from the agent’s per-action path. Validation of padded costs does not optimize compressed archives or model tokenization; adoption must measure the actual delivery representation.

5 Practical Online Selection: Price the Window, Not the Promise

Freeze a compiled catalogue $C = \{L_1, \dots, L_K\}$. At event t , choose a layout before observing the current completion wave; afterwards score that same wave under every catalogue member. This is full-information feedback only because the check schedule is held fixed. It is not counterfactual feedback for arbitrary agent behavior.

A different layout is installed by serializing its canonical manifest and reconstructing its complete certificate tree:

$$M_{ij} = \begin{cases} 0, & L_i = L_j, \\ |\text{manifest}(L_j)| + W(L_j), & L_i \neq L_j. \end{cases} \quad \mathcal{L}_T = \sum_t \ell_t(A_t) + \sum_{t=2}^T M_{A_{t-1}, A_t}. \quad (12)$$

The initial installation is separate. These byte prices match the evaluated uncompressed migration protocol. A deployment using compression must measure its own priced wire representation; the DP objective is not automatically an optimum for a compressed log.

The primary practical rule. Let $\bar{\ell}_{t,i}$ be the mean cost on the last $\min(w, t-1)$ observed events, and $j_t = \arg \min_i \bar{\ell}_{t,i}$ with deterministic ties. Starting from the calibrated incumbent a , use the frozen v3 rule

$$A_t = \begin{cases} j_t, & w(\bar{\ell}_{t,a} - \bar{\ell}_{t,j_t}) > M_{a,j_t}, \\ a, & \text{otherwise,} \end{cases} \quad w = 64. \quad (13)$$

With no history, keep a . A tie in estimated savings does not trigger migration. During warm-up, the forecast horizon remains w even if fewer events were observed, matching the original baseline. This is an empirical break-even forecast, not proof that past savings repay a future switch. It can lose under abrupt reversal; preserving the frozen layout is always a legitimate alternative.

The runtime API enforces `choose()` before `observe()`, bounds history to 64 vectors, rejects nonfinite costs, and checkpoints only at event boundaries. Replaying all 80 archived v3 streams, with periodic checkpoint/resume, must reproduce the original baseline’s full action and byte-cost paths exactly. The exercise validates implementation equivalence, not a new performance dataset.

Why fixed-share is no longer primary. On v3’s stationary, long-shift and rapid-shift streams, the coupled fixed-share policy is more expensive than the priced window. Its standard tracking analysis therefore moves to Appendix A, with all negative results retained. We do not manufacture an adversarial workload merely to make that policy win. Neither the appendix bound nor the empirical window result establishes general superiority over other switching-cost policies.

6 Stable Evidence Through Permutation and Migration

A result token carries (epoch, g_i, b_i) , where b_i binds the semantic ID to the registry, source snapshot, checker, and environment. It does not carry a display position as its identity. A layout-specific kernel

Algorithm 1 Deployable evidence control; inference and checking remain separate**Require:** Protected registry, checker and snapshot identities; frozen layout catalogue

```

1: for each event do
2:   Choose from past evidence using Equation (13), or keep the frozen layout
3:   if layout changes then
4:     Revoke tickets, increment generation, remap stable IDs, rebuild and price the installation
5:   end if
6:   Receive trusted completion records for the fixed check schedule
7:   Reject stale, conflicting or malformed records; aggregate bounded packets
8:   Publish only the complete current contract result; otherwise retain FAIL/UNKNOWN
9:   Expose a root-status card; retrieve diagnostic packets only when needed
10:  Score the observed event and append its cost vector to the bounded window
11: end for

```

maps stable IDs to current positions. During migration it rebuilds from canonical trusted records; it never transplants an ordinal cache entry into a different permutation.

Publication tickets additionally carry a monotone layout generation and layout hash. A layout switch revokes the issued ticket, advances the generation, and forces reconstruction. Returning to an earlier tree therefore cannot revive that tree’s old ticket. An in-flight token may remain valid through a layout-only change, but a new candidate epoch fences it. Duplicate identical evidence is idempotent. Conflicting final records in one epoch revoke authorization: a recorded fail remains a fail; otherwise the obligation becomes unknown. Resolution requires a new epoch, not a more persuasive agent summary.

Theorem 5 (Layout-independent local publication soundness). *Assume complete approved predicates and dependency identities, a trusted checker executing immutable identified inputs, collision-resistant binding, and atomic local epoch/generation checks. For any sequence of valid layout permutations, any advisory learning outputs, and any delivered evidence order, authorization implies Equation (2). Moreover, two valid layouts reconstructed from the same canonical record set have identical root pass/fail/unknown counts and therefore the same semantic status projection. If each checker is sound for its declared predicate, the declared predicates hold on the authorized candidate.*

Proof. A canonical pass requires a current token for the same semantic ID and immutable identities. Permutation is a bijection, so rebuilding moves each such record to exactly one position without changing its meaning. Exact child partitioning preserves coverage and pass/fail/unknown counts by induction up the tree. The root can be authorized only with $(n, 0, 0)$ and matching current registry, epoch, generation, and layout. Old tickets fail those checks. Conflicts prevent authorization. No choice made by the learner writes trusted outcomes or changes the registry. $\square$

This is a conditional local guarantee, not remote attestation or distributed consensus. An incomplete suite, compromised checker, omitted dependency, or nondeterministic external service can invalidate a broader correctness claim. The CLI’s file copy is not an OS sandbox. An external merge/deployment service must atomically compare the verified candidate and integration-head identities at commit; a report file alone is not that transaction.

7 Implementation and Cost Accounting

The v5 artifact adds `practical.py`, `deployment.py`, `token_budget.py` and `tls_probe.py`. The practical CLI produces a balanced catalogue without calling either exact or block DP. The historical exact and block compilers remain separately callable. The existing stable-ID guard, priced window, codec and Future Code bridge are retained. The strict CLI compares protected input *sets* and hashes, detecting added tests or configuration files as well as changed and deleted ones. Schema-2 contracts bind the approved

Table 1: Separate ledgers avoid converting byte savings into unsupported latency or token claims.

Measurement	How obtained	Included in optimized objective?
Certificate bytes	Actual padded packets; independently reconstructed from waves	Yes
Migration bytes	Canonical layout manifest plus forced full rebuild	Yes, online traces
Fit and scoring time	Monotonic-clock measurements	Reported separately
Preparation, copying, checking, aggregation	Wrapper timers and outer call duration	Reported separately
Active harness episode	Supervisor startup through task completion and shutdown	Reported separately
Prompt bytes and construction time	Captured actual HTTP messages and context builder	Reported; not tokens
Hosted token usage and API billing	No provider inference was run	Unknown, never zero-imputed

checker and environment and refuse silent reuse of legacy contracts. Source manifests are checked before and after snapshotting and verification; mutation of an observed snapshot prevents a pass.

The bridge runs as an ordinary subprocess acceptance gate in the unmodified Future Code v1.2 runtime. The supervisor still owns worker leases, private attempts, bounded contexts, and serialized integration. Non-pass exits cause candidate rejection. Detailed evidence and binary certificate packets live outside candidate write scopes; only a bounded JSON result card returns to the worker. The plug-in accepts a precompiled layout. The online selector and migration guard are separately integrated and stress-tested as a library path; Section 10 states the remaining end-to-end deployment gaps.

A different scalar objective could combine these quantities only with declared unit conversions and observable counterfactual costs. This implementation does not optimize unknown model latency or billing. The exact-byte claim also depends on the stated padded format; compression defines a different, potentially nonseparable objective for which this DP is not claimed exact.

8 Service-Matched Delivery: What HACT Can Save

8.1 Three consumers define three services

The local aggregator receives bounded packets; a logging collector may receive their complete stream; a supervisor normally receives a status projection and retrieves details selectively. Define $\psi(e, c)$ as canonical status text containing semantic identity, counts and local authorization, but no display positions. For the same trusted semantic report,

$$\psi_{L_1}(e, c) = \psi_{L_2}(e, c). \quad (14)$$

Its byte length and token count under any fixed deterministic tokenizer are therefore identical. This is a useful negative control: layout optimization cannot shrink a layout-independent root input. A flat ledger similarly provides final per-check states without preserving the complete intermediate hierarchy. HACT’s layout objective applies when the hierarchical diagnostic stream itself is part of the required service.

Diagnostics preserve full header/card records and page whole packets under a 16,384-byte limit. A packet that cannot fit is rejected rather than truncated. For a real supplied tokenizer τ and explicitly reserved framing budget r , token-aware admission additionally requires

$$|p|_{\text{UTF8}} \leq C_{\text{bytes}}, \quad |\tau(\text{decode}_{\text{UTF8}}(p))| + r \leq C_{\text{tokens}}. \quad (15)$$

The implementation retokenizes each complete proposed page, including its prefix; it does not assume additivity of fragment token counts. The optional measurement command requires an actual `tiktoken`

encoding [19]. Provider framing and billed usage remain properties of a provider adapter, not of the byte objective.

8.2 Codec boundaries are part of the service

Canonical unpadded JSON, independently compressed packets, and whole-epoch DEFLATE preserve the same logical records but expose different streaming and compression behavior. The padded objective is therefore not claimed to optimize compressed bytes or model tokens exactly. Practical selection is advisory until the deployed representation is measured with the deployed codec.

This distinction also separates representation choice from verification choice. If a deployment already requires protected hierarchical verification, fixed and learned layouts can be compared at marginal cost under the same service. If the baseline used ordinary testing with no protected evidence service, adopting the verifier is a separate engineering decision and its additional cost cannot be charged to, or justified by, layout compression alone.

8.3 Break-even accounting

Consider q copies of the *same* workload and delivery service through one nonoverlapped bottleneck. Let B_0, B_1 be codec-matched application bytes per copy, $R > 0$ effective application goodput, $E \geq 0$ additional encoding/decoding work per copy, and $C \geq 0$ consistently amortized one-time incremental work. Common setup cancels only when it is actually common. The conditional net benefit is

$$d = (B_0 - B_1)/R - E, \quad \Delta(q) = qd - C. \quad (16)$$

A strictly profitable integer copy count exists only when $d > 0$, and then

$$q_{\min} = \lfloor C/d \rfloor + 1. \quad (17)$$

A zero gain retains the baseline. The implementation uses decimal arithmetic at equality, rejects mismatched service IDs, and returns UNKNOWN when required measurements are absent. An explicit zero is accepted only as a declared assumption or observation.

When both variants use the same protected gate, C represents extra layout work rather than the entire gate cost. Repeating delivery can amortize preparation of the same candidate; it cannot amortize preparation of future candidates as though they were the same workload. Parallel links, computation overlap, cached diagnostics, queueing, and connection setup require direct critical-path measurement. RTT alone does not increase the byte-dependent term when both alternatives pay the same setup and request count.

8.4 Remote-measurement boundary

The artifact includes an opt-in TLS measurement kit with certificate validation, hostname checking, bounded frames, validated decompression, and acknowledgments that bind received length and digest. Local tests exercise cold and reused sessions, rejected hostnames, untrusted roots, malformed acknowledgments, and truncated records. These tests validate the measurement path; they are not WAN performance measurements.

A qualifying remote experiment must record host/path provenance, codec, connection policy, matched payloads, application goodput, and transport observations such as RTT or retransmissions where available, following the separation between application payload and transport behavior emphasized in TCP throughput testing [21]. TLS authenticates the configured server, not the truthfulness of a checker. Attestation, Byzantine consensus, client authorization, non-repudiation, and OS isolation remain outside HACT's local semantic guarantee.

9 Evaluation

9.1 Questions, frozen protocols, and units of analysis

The journal evaluation separates six questions:

Table 2: Evaluation questions and the evidence used to answer them.

RQ	Question	Evidence	Primary unit
RQ1	Does HACT preserve verification semantics under layout changes?	Theorem 5; 36 three-layout source replays; root-status equality	candidate/layout replay
RQ2	Does learned ordering replicate, and does it beat weaker ordering baselines?	Precommitted 180-workload disjoint-seed study; marginal and validated-random baselines; latent-order reference	independent seed-level workload
RQ3	What happens when no exploitable completion locality exists?	Independent and global negative controls; validated and unvalidated learned proposals	independent seed-level workload
RQ4	How sensitive is a learned layout to distribution shift?	Precommitted 60-workload latent-permutation shift study with stale and retuned layouts	independent seed-level workload
RQ5	Does the cost signal occur in source-derived evidence?	Previously frozen 33-case source confirmation set plus 12-case codec replay	held-out source candidate
RQ6	When can representation savings justify extra work?	Codec-matched bytes and 27 declared deployment scenarios	service scenario

The original v5 contract used three seeds per size/family and motivated the journal extension. Before running the extension, we committed `V6_JOURNAL_EXPERIMENT_CONTRACT.json`, fixing hypotheses, twenty new seeds per cell, baselines, train/validation/held-out sizes, the distribution-shift protocol, bootstrap procedure, and non-goals. Its SHA-256 is `ef6b8ff8ff102228...abbc8625`. No seed, baseline, threshold, or primary metric was replaced after outcomes were observed.

For each ordinary workload, 128 training epochs fit the proposal, 128 disjoint validation epochs choose between a proposal and the incumbent, and 256 disjoint held-out epochs determine the reported cost. The statistical unit is the *seed-level workload*, not an individual epoch. We report means, medians, ranges, percentile 95% bootstrap intervals of the seed-level mean (10,000 fixed-seed resamples), and exact one-sided sign tests for directional replication. These intervals characterize the frozen generator family; they are not population intervals for all software projects.

9.2 RQ1: semantic output is independent of valid evidence layout

Theorem 5 establishes the conditional semantic property. The source replay provides an implementation-level cross-check: semantic pass/fail/unknown counts, verdicts, and local authorization agree across all 36 layout replays (12 candidates under fixed8, learned balanced, and learned exact). Root-status text is byte-identical in every layout triple and occupies 593–594 UTF-8 bytes. Thus the measured representation changes affect hierarchical diagnostics while leaving the declared root-status service unchanged.

9.3 RQ2: disjoint-seed replication and baseline adequacy

The journal replication contains 180 ordinary workloads: three registry sizes, three completion families, and twenty seeds per cell. Table 3 reports clustered completions, where the generator contains stable eight-check locality unknown to the learner.

Table 3: Precommitted disjoint-seed replication on clustered completion waves. Entries are mean held-out padded-cost reduction versus original-order fixed8, with 95% bootstrap intervals of the seed-level mean in brackets. The latent reference uses the generator’s hidden group order with the same balanced tree; it is a diagnostic reference, not a mathematical upper bound.

Checks	Validated HACT	Marginal	Validated random	Latent ref.	HACT regress.
128	23.56 [23.20,23.94]	15.63 [14.89,16.39]	1.17 [0.68,1.70]	23.55 [23.20,23.93]	0/20
512	10.66 [10.42,10.91]	4.90 [4.76,5.03]	0.23 [0.14,0.32]	12.42 [12.34,12.49]	0/20
1,024	5.26 [5.16,5.37]	2.01 [1.96,2.05]	0.05 [0.01,0.09]	8.06 [8.03,8.09]	0/20

Validated HACT improves all twenty seeds at every size. Mean reductions are 23.56%, 10.66%, and 5.26%; the corresponding one-sided sign-test value is 9.54×10^{-7} at each size. The three 95% bootstrap intervals remain well above zero. The original three-seed v5 estimates (23.43%, 10.75%, 5.56%) lie close to the new means, so the larger study replicates rather than reverses the earlier effect.

The baseline comparison also clarifies what is learned. Sorting by marginal completion frequency preserves some locality but reaches only 15.63%, 4.90%, and 2.01% mean reduction. Validation over eight random permutations gives 1.17%, 0.23%, and 0.05%. The learned-versus-marginal mean gap is therefore 7.93, 5.76, and 3.26 percentage points across the three sizes; the learned-versus-random gap is 22.40, 10.43, and 5.21 points. At 512 and 1,024 checks the latent-order reference remains better than the learned order, quantifying remaining ordering headroom. At 128 checks their means are nearly equal and the learned mean is slightly larger; hence the latent order is not called an oracle or upper bound.

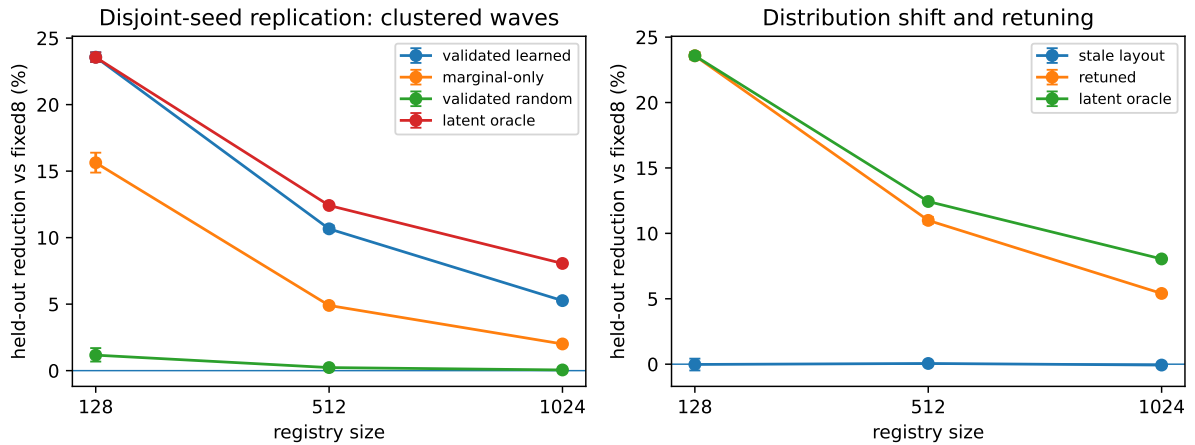


Figure 2: Journal extension. Left: disjoint-seed clustered replication against marginal, validated-random, and latent-order references. Right: a latent cluster-permutation shift makes the old layout uninformative, while retuning on post-shift evidence recovers most of the structured benefit. Error bars are 95% bootstrap intervals of the seed-level mean.

The practical path invokes no tree DP. Median fitting time is approximately 0.02, 0.63, and 2.47 seconds at 128, 512, and 1,024 checks in this run. Fitting therefore remains control-plane work rather than an action-loop operation.

9.4 RQ3: negative controls delimit the useful regime

Table 4: Validated HACT on null controls. Changed counts validation decisions that selected the learned proposal; regress. counts held-out reductions below zero. Confidence intervals spanning zero are evidence against a systematic benefit, not evidence of equivalence.

Checks	Family	Mean (%)	95% CI	Changed	Regress.
128	independent	-0.043	[-0.119,0.022]	11/20	6/20
128	global	0.000	[0.000,0.000]	0/20	0/20
512	independent	-0.010	[-0.041,0.017]	6/20	3/20
512	global	0.000	[0.000,0.000]	0/20	0/20
1,024	independent	-0.013	[-0.031,0.002]	9/20	6/20
1,024	global	0.000	[0.000,0.000]	0/20	0/20

Global waves are analytically layout-insensitive under the balanced representation, and all sixty global replications yield exactly zero. Independent waves are more informative: means are -0.043%, -0.010%, and -0.013%, with all three bootstrap intervals spanning zero. Validation often retains the incumbent, but it does not eliminate sampling error: the learned proposal is selected in 11/20, 6/20, and 9/20 seeds and produces 6, 3, and 6 small held-out regressions respectively. The unvalidated learned proposal regresses in 10/20, 9/20, and 10/20 independent seeds, so the hold-out guard avoids 4, 6, and 4 regressions without turning the selection procedure into a zero-regret guarantee.

This result sharpens the deployment rule: HACT needs stable higher-order locality, and validation is a risk-reduction device rather than proof that adaptation can never hurt.

9.5 RQ4: distribution shift requires retuning

The shift study uses twenty additional, disjoint seeds at each size. Training/validation regime A and post-shift regime B use independent latent permutations of the same clustered generator. The stale policy is chosen using only regime-A validation; the retuned policy is refit and selected only on regime-B training/validation, then both are evaluated on regime-B held-out data.

Table 5: Precommitted latent-permutation shift. Entries are mean reduction versus fixed8 with 95% bootstrap intervals. The stale and retuned columns use the same validation/fallback rule in their respective regimes.

Checks	Stale layout	Retuned	Latent ref.	Stale regress.	Retuned regress.
128	-0.02 [-0.48,0.42]	23.58 [23.30,23.84]	23.57 [23.30,23.85]	9/20	0/20
512	0.05 [-0.07,0.17]	10.99 [10.73,11.26]	12.43 [12.36,12.50]	9/20	0/20
1,024	-0.07 [-0.10,-0.03]	5.41 [5.30,5.53]	8.04 [8.02,8.07]	15/20	0/20

A stale learned layout provides essentially no mean benefit after the latent permutation changes: -0.02%, 0.05%, and -0.07% at 128, 512, and 1,024 checks. At 1,024 checks the bootstrap interval is entirely below zero (-0.10 to -0.03). Retuning restores 23.58%, 10.99%, and 5.41% reductions, is positive in all sixty shifted replications, and shows no held-out regression. The retuning gains are 23.60, 10.94, and 5.47 percentage points; each size has a one-sided sign-test value of 9.54×10^{-7} . This validates the paper’s decision to price migration and treat adaptation as a controlled process rather than a permanent learned ordering.

9.6 RQ5: source-derived evidence supports, but does not duplicate, the synthetic result

Two source studies answer different questions. First, a previously frozen confirmation cohort partitions archived cases into training, validation, and held-out confirmation sets for NetworkX [11], Toolz [12], and

Future Code. That older study uses the research exact conditional compiler, not the v6 practical balanced compiler.

Table 6: Previously frozen source confirmation for the selected exact conditional layout versus original-order fixed8. Percentages are computed per held-out case from recorded source evidence.

Project	Checks	Cases	Mean (%)	Median (%)	Positive
NetworkX	170	9	22.23	3.07	6/9
Toolz	148	6	11.90	9.58	6/6
Future Code	138	18	22.13	5.53	16/18
All	—	33	20.30	5.73	28/33

Across 33 held-out source cases, 28 improve and five regress; the exact one-sided sign-test value for the 28/33 direction count is 3.31×10^{-5} . The per-case range is wide (-7.33% to 63.56%), showing that workload structure matters substantially. This cohort supports the existence of layout-sensitive structure in source-derived verification waves, while its different compiler and older data contract prevent us from presenting it as a replication of the practical v6 effect.

Second, the v5 codec replay re-aggregates twelve archived candidates under original fixed8, learned balanced8, and learned exact layouts. Learned balanced reduces whole-epoch compressed bytes by 41.84% for NetworkX, 4.36% for Toolz, and 18.53% for Future Code. Across the twelve cases, batch-compressed application bytes are 416,046 for fixed8, 299,081 for learned balanced, and 278,831 for learned exact. Exhaustive diagnostic pages total 148, 95, and 87 respectively. Thus the practical layout retains real compression/page benefits while the exact reference quantifies additional headroom.

9.7 RQ6: deployment economics remain service-specific

The measured fixed-to-balanced compressed-byte difference is 116,965 bytes per twelve-candidate cohort. Under an explicitly assumed 64 KiB/s application goodput and $E = 0$, Equation (16) maps that byte difference to 1.784744 seconds of gross delivery time per cohort copy. This is arithmetic from measured application bytes, not a measured WAN speedup.

Table 7: Conditional minimum profitable copies of the same twelve-candidate cohort for whole-epoch compression, assuming $E = 0$. Column headers are extra work per candidate; corresponding one-time cohort costs are 0.6, 6, and 60 seconds.

Assumed goodput	0.05 s	0.5 s	5 s
8 KiB/s	1	1	5
64 KiB/s	1	4	34
1 MiB/s	6	54	538

The artifact stores 27 codec/rate/extra-cost scenarios and retains negative outcomes. A deployment should therefore decide that it needs hierarchical evidence first, measure the actual codec and bottleneck second, and only then decide whether a learned layout repays its fitting/migration work.

9.8 Reproducibility, independent reconstruction, and failure provenance

The journal extension stores 180 ordinary and sixty shift workload records, exact seed sets, layout orders, validation/held-out totals, dataset hashes, environment metadata, generated tables, and Figure 2. An independent auditor regenerates each synthetic dataset, reconstructs compact-tree packet coverage without calling the producer’s cost aggregation helper, recomputes every validation/held-out byte total, verifies the

contract digest, and includes a deliberate tamper test. The final audit reports PASS for 180 ordinary and sixty shifted workloads and PASS for tamper detection.

The first audit attempt failed because the auditor incorrectly charged cold initialization once per cohort rather than once per epoch. No results were committed from that failed run. The accounting identity was corrected in the auditor only; the frozen contract, producer, seeds, baselines, and thresholds were unchanged. The rerun passed. Preserving this failure is useful evidence that the independent path was capable of rejecting a producer/auditor disagreement rather than merely echoing producer outputs.

10 Implications and Threats to Validity

Operational implication. HACT should be selected only after the evidence service is selected. For a trusted final verdict, the root projection is the natural representation; for final per-check states, a flat ledger may suffice. When a system already requires bounded hierarchical diagnostics, learned-order balanced8 is a low-compilation-cost candidate and the incumbent remains a first-class fallback. The journal replication makes the condition more precise: stable clustered completion structure produces a large and repeatable benefit, whereas independent and global controls do not. A deployment should therefore cache layouts, measure drift, and retune only when new evidence justifies the migration price.

Validation reduces risk; it does not create a no-regret selector. The independent controls are important because they falsify a stronger, tempting claim. Even with a separate validation cohort, sampling variation can select a learned layout that is slightly worse on held-out independent waves. The observed mean effects are near zero and their bootstrap intervals cross zero, but 6/20, 3/20, and 6/20 seeds regress at the three sizes. Validation avoids 4, 6, and 4 regressions relative to unconditional deployment of the learned proposal; it does not eliminate them. Production use therefore needs an economic switching threshold and, where regression tolerance is strict, larger validation evidence or an explicit statistical/non-inferiority guard.

Distribution shift is a first-class systems concern. When the latent cluster permutation changes, the previously useful layout loses essentially all benefit and is slightly harmful on average at 1,024 checks. Retuning restores a positive result in all sixty shifted replications. This supports the architectural separation between a stable semantic registry and a mutable layout plane: adaptation is valuable precisely because layout state can become stale while the verification contract remains unchanged. It also argues against continuously self-modifying the acceptance semantics in response to the same telemetry.

Construct validity. The primary objective is serialized hierarchical-evidence cost under an explicit packet model. Padded bytes, canonical bytes, compressed bytes, diagnostic pages, model tokens, wall-clock latency, and end-to-end agent productivity are different quantities. We measure the first four where available and do not convert bytes to tokens by a heuristic ratio. The break-even model uses declared goodput and incremental work; sensitivity scenarios are planning inputs rather than measured WAN latency. Whole-epoch compression changes streaming semantics, so a smaller archive is not automatically a better online service.

Internal and statistical validity. The journal extension was precommitted before execution, uses disjoint train/validation/held-out draws, and treats each seed-level workload as one statistical unit. Twenty seeds per cell are a substantial improvement over the original three, but the bootstrap intervals still describe this frozen generator rather than an unrestricted software population. Directional sign tests support the predeclared clustered/retuning hypotheses; they are not used as a post-hoc discovery screen over every reported metric. The failed first independent audit is retained in provenance: it exposed an error in the verifier's initialization accounting, no failed-run result was committed, and the correction changed the verifier only. The final independent recomputation and tamper test pass.

External validity. The synthetic generator deliberately isolates completion locality, null regimes, scale, and a controlled permutation shift. The separate source-derived evidence spans NetworkX, Toolz, and Future Code: 33 held-out confirmation cases show 28 positive exact-layout results, and twelve practical-layout codec replays show compressed-byte/page reductions. These cohorts broaden the evidence beyond synthetic waves, but they still cover only three software systems and reuse archived verification records. They do not measure a new natural-bug population, live model-generated repairs, provider billing, or physical-WAN behavior. A stronger future external-validity study should freeze a multi-repository natural-history benchmark and run the same semantic contract, codec, and layout policies on newly collected candidate streams.

Security and semantic boundary. Theorem 5 is conditional on a complete approved predicate set, correctly bound dependencies, a trusted checker, collision-resistant identity binding, and atomic local epoch/generation checks. HACT does not provide remote attestation, Byzantine consensus, non-repudiation, or an OS sandbox. A separate integration service must atomically bind the verified candidate to the integration head before merge or deployment. Evidence-layout independence protects the meaning of the declared contract; it cannot make an incomplete contract complete.

Reproducibility and artifact scope. The repository retains the frozen journal contract, producer, independent auditor, exact seed sets, environment metadata, per-workload orders and totals, hashes of deterministic generated waves, generated tables, and journal figure. Historical v3–v5 outputs remain versioned rather than rewritten into the new cohort. This separation allows reviewers to distinguish reproduction of old claims, new journal-extension evidence, and historical controls.

11 Conclusion

Adaptive HACT turns hierarchical verification evidence into a workload-adaptive representation problem under a semantic invariant. Stable identities and generation fencing separate authorization from layout; completion hyperedges provide the signal needed to exploit higher-order locality without allowing the learner to write trusted outcomes.

The precommitted journal extension strengthens the empirical claim substantially. Across sixty clustered replications, validated HACT improves every seed, with mean held-out padded-cost reductions of 23.56%, 10.66%, and 5.26% at 128, 512, and 1,024 checks and narrow seed-level bootstrap intervals. Marginal-frequency and random-order baselines are materially weaker. Null controls show no systematic benefit and reveal that validation is risk-reducing rather than no-regret. Under a controlled distribution shift, stale layouts lose their advantage while retuning restores 23.58%, 10.99%, and 5.41% mean reductions with no observed regressions. Previously frozen source-derived confirmation and codec studies show that layout-sensitive cost is not confined to the synthetic generator, while also exposing substantial case-to-case variation.

These results support a specific systems claim: when hierarchical verification evidence is required and completion structure is stable enough to learn, HACT can reduce its representation and delivery cost while preserving the declared acceptance semantics. They do not establish universal gains in agent intelligence, token use, WAN latency, or software quality. The separation between semantic safety, workload-sensitive representation, drift-aware adaptation, and deployment economics is the paper’s central contribution.

References

- [1] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929–948, 2001. <https://doi.org/10.1109/32.962562>.

- [2] Milos Gligoric, Lamyaa Eloussi, and Darko Marinov. Practical regression test selection with dynamic file dependencies. *Proceedings of ISSTA*, 2015.
<https://users.ece.utexas.edu/~gligoric/papers/GligoricETAL15Ekstazi.pdf>.
- [3] You Wang, Michael Pradel, and Zhongxin Liu. Names Are All You Need: Effective and Safe Regression Test Selection for Python. arXiv:2605.25356v1, May 2026.
<https://arxiv.org/abs/2605.25356v1>.
- [4] Umut Acar, Guy Blelloch, Matthias Blume, Robert Harper, and Kanat Tangwongsan. A Library for Self-Adjusting Computation. *Electronic Notes in Theoretical Computer Science*, 148:127–154, 2006.
<https://doi.org/10.1016/j.entcs.2005.11.043>.
- [5] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build Systems à la Carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP), Article 79, 2018.
<https://doi.org/10.1145/3236774>.
- [6] Hirendu Vaishnav and Massoud Pedram. Alphabetic Trees: Enumeration and Optimization with Applications to VLSI Design. Author-hosted manuscript, Sections 4.1–4.4; date not established from the retrieved manuscript.
<https://www.mpedram.com/Papers/alp-trees-journal.pdf>.
- [7] Ziheng Shangguan, Aviv Yaish, and Dahlia Malkhi. Authenticated Data Structures for Dynamic Workloads. arXiv:2608.25206v1, August 2026.
<https://arxiv.org/abs/2608.25206v1>.
- [8] Zexun Wang. Proof-Carrying Agent Actions: Model-Agnostic Runtime Governance for Heterogeneous Agent Systems. arXiv:2606.04104v1, June 2026.
<https://arxiv.org/abs/2606.04104v1>.
- [9] Zihao Zheng, Baichuan Li, Junyi Yao, and Jiayu Long. Engineering Reliable Commit Gates for Agentic AI: Cost-Aware Verification Portfolios under Common-Mode Data Failures. arXiv:2609.10969v1, September 2026.
<https://arxiv.org/abs/2609.10969v1>.
- [10] Wassily Hoeffding. Probability Inequalities for Sums of Bounded Random Variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.
<https://doi.org/10.1080/01621459.1963.10500830>.
- [11] NetworkX developers. NetworkX 3.6.1 release documentation and distributed source tests.
https://networkx.org/documentation/stable/release/release_3.6.1.html.
- [12] PyToolz contributors. Toolz API documentation and distributed source tests.
<https://toolz.readthedocs.io/en/stable/api.html>.
- [13] Jonathan E. Atkins, Erik G. Boman, and Bruce Hendrickson. A Spectral Algorithm for Seriation and the Consecutive Ones Problem. *SIAM Journal on Computing*, 28(1):297–310, 1998.
<https://doi.org/10.1137/S0097539795285771>.
- [14] Mark Herbster and Manfred K. Warmuth. Tracking the Best Expert. *Machine Learning*, 32:151–178, 1998.
<https://doi.org/10.1023/A:1007424614876>.
- [15] Sascha Geulen, Berthold Vöcking, and Melanie Winkler. Regret Minimization for Online Buffering Problems Using the Weighted Majority Algorithm. *Proceedings of COLT*, pages 132–143, 2010.
<https://www.learningtheory.org/colt2010/conference-website/papers/105geulen.pdf>.

- [16] Nicolò Cesa-Bianchi, Ofer Dekel, and Ohad Shamir. Online Learning with Switching Costs and Other Adaptive Adversaries. *Advances in Neural Information Processing Systems*, 26, 2013.
<https://papers.neurips.cc/paper/5151-online-learning-with-switching-costs-and-other-adaptive-adversaries.pdf>.
- [17] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations*, 2024.
<https://arxiv.org/abs/2310.06770>.
- [18] OpenTelemetry contributors. Batch Processor documentation, official collector implementation. Accessed September 20, 2026.
<https://github.com/open-telemetry/opentelemetry-collector/tree/main/processor/batchprocessor>.
- [19] OpenAI contributors. tiktoken: BPE tokenization implementation and encoding registry. Accessed September 21, 2026; package/vocabulary unavailable in the v5 measurement runtime.
<https://github.com/openai/tiktoken>.
- [20] SWE-bench contributors. The Harness: Docker-based patch evaluation reference. Accessed September 20, 2026.
<https://www.swebench.com/SWE-bench/reference/harness/>.
- [21] B. Constantine, G. Forget, R. Geib, and R. Schrage. Framework for TCP Throughput Testing. RFC 6349, August 2011.
<https://www.rfc-editor.org/rfc/rfc6349.html>.

A Optional Fixed-Share Analysis: Not the Deployment Recommendation

This section retains v3’s former Theorem 4 and its assumptions. It is an optional comparator, not the practical selector recommended in v4. On the measured streams it loses to the deterministic priced-window rule in three regimes. We have not searched for a favorable adversarial stream after seeing those results, and make no claim that a tracking bound implies an empirical advantage.

A.1 Fixed-share weights with maximal coupling

Let a declared bound R cover each round’s loss spread, $\max_i \ell_t(i) - \min_i \ell_t(i) \leq R$. Reject a measurement exceeding it instead of clipping the cost. Define $z_{t,i} = (\ell_t(i) - \min_j \ell_t(j))/R$. Starting at $p_1(i) = 1/K$, update

$$q_{t+1}(i) = \frac{p_t(i) \exp(-\eta z_{t,i})}{\sum_j p_t(j) \exp(-\eta z_{t,j})}, \quad (18)$$

$$p_{t+1}(i) = (1 - \alpha)q_{t+1}(i) + \alpha/K, \quad 0 < \eta \leq 1, \quad 0 \leq \alpha < 1. \quad (19)$$

Independent draws from each new distribution can waste migrations. Instead, conditional on $A_t = i$, retain i with probability $\min\{1, p_{t+1}(i)/p_t(i)\}$. If it is not retained, sample from the normalized positive residual $(p_{t+1} - p_t)_+$. This maximal coupling has marginal p_{t+1} and switch probability $\text{TV}(p_t, p_{t+1})$. Established dependent-sampling methods motivate this choice [15]; we do not claim it as a new sampling algorithm.

Theorem 6 (Catalogue tracking with migration). *Assume a frozen catalogue, a fixed exogenous sequence of full-information loss vectors, spread at most R , and $M_{ij} \leq M$. For any comparator path $u_{1:T}$ with s*

changes and $0 < \alpha < 1$, the coupled policy satisfies

$$\mathbb{E}[\mathcal{L}_T] - \sum_t \ell_t(u_t) \leq R \left[\frac{\log K + s \log(K/\alpha) + (T-1-s) \log(1/(1-\alpha))}{\eta} + \frac{\eta T}{8} \right] + M(T-1)(\eta + \alpha). \quad (20)$$

For $\alpha = 0$ the stationary-comparator bound has numerator $\log K$ and applies only to $s = 0$.

Proof. The fixed-share update is exponential weighting over paths with an initial uniform prior, self-transition mass at least $1 - \alpha$, and transition mass at least α/K . The negative log prior of a path with s switches is bounded by the numerator. The bounded exponential-moment inequality gives expected normalized service regret at most that numerator divided by η , plus $\eta T/8$ (Appendix D). Since losses are exogenous, the weights are independent of the sampled action path, and the coupling preserves each unconditional marginal. Exponential reweighting changes total variation by at most η ; uniform mixing adds at most α . Thus the expected switch count is at most $(T-1)(\eta + \alpha)$. Multiplication by M and addition of service regret proves the bound. $\square$

The bound also upper-bounds regret relative to the comparator’s service *plus* nonnegative migration cost, but can be quite loose. It is not a per-run improvement guarantee. Choosing fixed $\eta = .5$ and $\alpha = .01$, as in our frozen experiment, need not make this bound sublinear with increasing T . Horizon-dependent tuning yields sublinear stationary bounds (Appendix D), without creating a guarantee against arbitrary reactive agents or newly introduced catalogue entries.

B Block-Backbone Compiler: Retained Ablation

The exact compiler’s cubic cost makes frequent full recompilation inappropriate. For a supplied display order, divide the leaves into consecutive blocks of at most s leaves, and fix a compact b -ary macro-tree on those blocks. Optimize each block’s microtree using its restricted completion waves and strict-epoch initialization cost. Replace each macro leaf by its optimized microtree. No evaluation waves enter this process.

Proposition 7 (Exactness in the block-backbone class). *With display order, block boundaries and macro-tree fixed, independent exact optimization of the blocks minimizes the training objective over every layout in that restricted class. The dynamic-programming work is $O(bns^2)$ with $O(bs^2)$ reusable DP workspace, excluding order learning and wave preparation.*

Proof. Every macro node covers a fixed union of complete blocks, so its activation weight and packet price are constant across the class. All variable costs lie inside disjoint microtrees. The objective is a constant plus the sum of their independent objectives. Conditional exactness therefore gives the minimizing member. There are at most $\lceil n/s \rceil$ subproblems, each of size at most s , yielding the stated DP bound. Reusing the subproblem arrays yields the workspace bound. $\square$

For the reference implementation, M recorded waves and L total wave memberships add filtering and weight-preparation work of $O(\lceil n/s \rceil L + Mns)$, and storage for the input, output and local $M \times s$ arrays. This is not a linear-time claim for the entire pipeline. Dense affinity construction and order learning remain separate costs. No bounded approximation ratio against the unrestricted exact tree is asserted; held-out cost can deteriorate. The historical experiment froze $s = 32$ before measurement. Its negative result motivates excluding this compiler from the v5 practical path.

The original 18-workload v4 scaling data remain in the artifact. Block DP reduced fitting time relative to exact DP but lost to same-order balanced fixed8 in held-out bytes. Restricted-class exactness is a mathematical property, not an empirical recommendation.

C Information Separations Retained from Earlier Versions

These results concern the information available to a selector. They are not hardness results for all adaptive tree algorithms, and they do not imply that a complete hypergraph must be recovered: interval hit statistics are sufficient for a fixed order and objective.

C.1 Marginals do not determine a co-invalidated optimum

Consider four ordered leaves, binary trees, and a constant internal packet price $c > 0$. Let $\mathcal{D}_A$ choose $\{0, 1\}$ or $\{2, 3\}$ equiprobably and $\mathcal{D}_B$ choose $\{0, 3\}$ or $\{1, 2\}$ equiprobably. All leaf marginal activation probabilities equal $1/2$ in both distributions. The root and every three-leaf interval are always activated. The five tree shapes therefore fall into the following classes:

Ordered tree	J_A/c	J_B/c
$((0, 1), (2, 3))$	2	3
$(0, ((1, 2), 3)), ((0, (1, 2)), 3)$	3	2.5
$(0, (1, (2, 3))), (((0, 1), 2), 3)$	2.5	3

Optimal costs are $2c$ and $2.5c$. A deterministic marginal-only selector receives identical input and must choose the same class, incurring at least $c/2$ regret on one distribution. The last class is weakly dominated by the balanced tree. If a randomized selector chooses the balanced tree with probability q , its regrets are at least $(1 - q)c$ and $qc/2$. Their maximum is minimized at $q = 2/3$ and equals $c/3$, which is attainable for this instance. A fixed initialization price adds the same $3c$ to all binary shapes and does not alter the separation.

C.2 Joint invalidation does not determine progressive layout

Every epoch now invalidates all four checks, and execution order is $(0, 1, 2, 3)$ with singleton completion waves. In workload A , check 0 fails, so only its wave is executed. In workload B , check 3 fails, so all four checks execute. Initial invalidation is identical, even jointly.

Every binary tree has three internal nodes. Its cost in A is $(3 + \text{depth}(0))c$, minimized at $4c$ by a root-0 comb. Its cost in B is $(3 + \sum_i \text{depth}(i))c$, minimized at $11c$ by the balanced tree. The root-0 comb costs $12c$ in B , whereas the balanced tree costs $5c$ in A . The remaining shapes are weakly dominated. An invalidation-only randomized selector using a root-0 comb with probability p incurs regrets at least $(1 - p)c$ and pc , whose maximum is at least $c/2$. A deterministic selector incurs at least c . The example proves a separation between stale-set information and schedule-induced completion information; it does not prohibit a selector that observes both.

D Cost, Statistical, and Online Proof Details

D.1 Exact serialization accounting

On the first refresh in a strict epoch, every internal node emits one packet. Within a fixed tree and registry, the cache uses monotone descendant-generation stamps. A subsequent wave emits a node's packet exactly when at least one final record in its semantic coverage changed. Multiple completed descendants in one wave still induce one packet at that node. Summing initialization and wave contributions yields Equation (4). After migration, a new kernel remaps canonical IDs and the old ordinal cache is not reused; the entire cold rebuild is charged in Equation (12).

This argument concerns logical and actual serialized packet lengths. The separate v4 study measures controlled application-level export time, not production-network throughput. It does not include raw log streams, TLS framing, operating-system scheduling, or Python traversal costs. The reference kernel

validates and traverses trees and is not claimed to have logarithmic total runtime. Small contexts do not imply small total system state.

D.2 Full-refresh lower bound and global negative control

For an n -leaf tree with m internal nodes, the number of edges is $n + m - 1$, so $\sum_v d_v = n + m - 1$. Therefore

$$W(L) = S(n - 1) + (H + S)m \geq S(n - 1) + (H + S) \left\lceil \frac{n - 1}{b - 1} \right\rceil. \quad (21)$$

The inequality follows from $n - 1 = \sum_v (d_v - 1) \leq m(b - 1)$. At $n = 64$, $b = 8$, a complete eight-child construction attains the bound with $m = 9$ and $W = 27,648$ bytes. Thus all selected global-control layouts can tie an analytically minimal full refresh. Strict initialization followed by one global completion wave costs $2W$. This explains the exact negative control without assuming every balanced tree at every n attains the lower bound.

D.3 Uniform statistical control at a fixed order

For one activation wave per episode and a fixed registry order, let $M_n = n(n + 1)/2$ and

$$\epsilon = \sqrt{\log(2M_n/\delta)/(2m)}.$$

The interval hit indicators are bounded independent variables *across* independent episodes, though checks within an episode may be arbitrarily dependent. Hoeffding plus a union bound gives simultaneous interval-frequency error at most ϵ . For every feasible tree, $|\hat{J}(T) - J(T)| \leq W(T)\epsilon$. Inserting the empirical minimizer gives regret at most $[W(\hat{T}) + W(T^*)]\epsilon \leq 2(n - 1)C\epsilon$. This is the original HACT bound, retained for attribution. It is not automatically valid uniformly over arbitrary data-selected permutations; the separate held-out catalogue bound in Proposition 4 addresses that selection step.

For strict epochs with at most q completion waves, a loose range is $B = (q + 1)(n - 1)C$. Under the evaluated sequential eight-record flush rule with at most one final record per obligation, $q \leq \lceil n/8 \rceil$. A narrower observed range is not automatically a proved bound for an unobserved deployment distribution.

D.4 Maximal coupling and its switch count

Let p, p' be consecutive distributions. The probability mass that remains at index i under the coupling is $\min(p_i, p'_i)$. The remaining mass is distributed as $(p' - p)_+$, so the new marginal is exactly p' and

$$\mathbb{P}(A_{t+1} \neq A_t) = 1 - \sum_i \min(p_i, p'_i) = \text{TV}(p, p').$$

For exponential reweighting by $a_i = \exp(-\eta z_i)$ with $z_i \in [0, 1]$, the normalizer is at most one, hence $q_i \geq e^{-\eta} p_i$. Thus $\sum_i \min(p_i, q_i) \geq e^{-\eta}$ and $\text{TV}(p, q) \leq 1 - e^{-\eta} \leq \eta$. Mixing q with the uniform distribution by weight α changes total variation by at most α . The triangle inequality gives the switch bound in Theorem 6. Zero old mass is never the probability of a selected previous action in exact arithmetic; the implementation rejects inconsistent checkpoints.

D.5 Fixed-share path argument

A path $u_{1:T}$ with s changes has prior mass at least

$$P(u_{1:T}) \geq K^{-1}(\alpha/K)^s(1 - \alpha)^{T-1-s}.$$

The share update is the forward recursion for exponential weighting over such paths. If $Z_t = \sum_i p_t(i) \exp(-\eta z_{t,i})$, then the bounded exponential-moment inequality implies

$$\log Z_t \leq -\eta \langle p_t, z_t \rangle + \eta^2/8.$$

On the other hand, the cumulative mixture likelihood is at least the comparator path’s prior times its exponential loss. Combining the inequalities yields

$$\sum_t \langle p_t, z_t \rangle - \sum_t z_{t,u_t} \leq -\log P(u_{1:T})/\eta + \eta T/8.$$

Centering losses by their per-round minimum cancels in regret, and multiplying by R restores byte units. For a fixed exogenous cost sequence, each p_t is determined by past full-information observations independently of the sampled action sequence; maximal coupling preserves its marginal. Therefore expected sampled service equals the left-side weighted service. Adding at most M per expected switch proves Equation (20).

The result does not cover a cost sequence chosen in reaction to the coupled hidden action history. It also does not cover agents changing future task choices because a layout changed the text they read. These limitations are why the paper distinguishes a verification-plane cost experiment from an end-to-end learning claim.

D.6 Horizon dependence and break-even interpretation

For $K \geq 2$, $s = 0$, and $\alpha = 0$, write $B_T = RT/8 + M(T - 1)$. The bound is $R \log K/\eta + \eta B_T$. When $\sqrt{R \log K/B_T} \leq 1$, choosing that learning rate gives at most $2\sqrt{R \log K B_T}$ expected regret, which is sublinear for fixed R, M, K . This is an established style of switching-cost bound, not a new rate. With a known sparse switch budget, selecting a horizon-dependent share similarly prices tracking complexity. Our fixed-parameter experiments do not invoke such a tuned asymptotic guarantee.

A single empirical switch with estimated future saving d and anticipated remaining horizon h satisfies the familiar break-even inequality $hd > M_{ij}$. This is arithmetic, not a drift theorem: savings may reverse before h events. The included window rule uses its past 64 events as that estimate and is deliberately reported as a heuristic. All choices, including greedy choices that ignore migration when deciding, are charged their actual transition price during evaluation.

E Experimental Reproduction and Threats to Validity

E.1 Software and data identities

The archived v3 runs used Python 3.13.5 on Linux x86_64; exact installed package and platform records accompany emitted reports. Compilers were measured with `OPENBLAS_NUM_THREADS=1`. No accelerator is required. Vended upstream sources and tests are retained with their notices. Every fresh mutation is checked against its archived base-source hash, replacement span, and resulting source hash. This binds a result to a specific intervention, not a natural historical bug distribution.

For the archived v3 record, there are three named output categories: `historical/results_v2` contains previous observations; `results/v3` contains v3’s generated layouts, raw waves, online paths, fresh subprocess reports, and harness records; `paper/generated` contains tables regenerated from those files. Running reproduction in a fresh destination avoids overwriting the released record. The release manifest identifies the exact packaged files. The final extracted archive is tested again before delivery. New v4 transfers, compiler traces, contexts, summaries and audits are stored separately in `results/v4`; they do not change the number of earlier checker or harness executions. V5 adds `results/v5` for the practical catalogue, archived-source re-aggregation, conditional deployment accounting and failed BPE attempts. Neither a new layout nor a reproduced experiment is counted as another repaired candidate.

E.2 Costs not hidden by replay

Ordering measurements include strict initialization; online traces explicitly use persistent-state incremental service plus cold switch prices. These are different lifecycles. Source replay uses a fixed previous ranker; fresh paired executions hold that ranker constant. The I/O harness does not compare different model-generated solutions: it sends identical scripted patch content to the same real runtime under different gates. Its model endpoint produces no provider token-usage field, so absent usage remains null in the records. Prompt byte lengths are measured directly and never converted to tokens by a heuristic multiplier.

Timing rows report outer-wrapper or active-episode boundaries explicitly. The wrapper's internal total ends just before final report and packet-file writes; the fresh study additionally records the outer call duration including those writes. Fit, contract approval, and baseline preparation are setup costs, not silently amortized into a claimed end-to-end speedup. With only three harness repeats, ranges are more honest than a precise broad-population confidence claim.

E.3 Test adequacy, nondeterminism, and I/O

The fixture performs real disk and loopback I/O but is intentionally controlled. It is not a test of internet reliability, production SQLite contention, nondeterministic service responses, or all flaky-test policies. When a collected check is skipped, missing, or inconsistent, it is not accepted as a pass. Two contradictory final records in one epoch trigger conflict handling, and an operator must establish a new epoch rather than cherry-pick the favorable record. Actual source tests can themselves be incomplete; agreement with their oracle is only agreement with the declared acceptance contract.

F Artifact Usage and Integration Contract

A reviewed operator baseline is initialized outside candidate write scopes:

```
python -m ichact init --project /work/trusted-baseline \
  --contract /work/contracts/approved.json \
  --output /work/reports/baseline --test-path tests

python -m ichact verify --project /work/candidate \
  --contract /work/contracts/approved.json \
  --output /work/reports/candidate-001 \
  --layout /work/contracts/layout.json
```

A layout must match the full semantic registry. The optional execution-order file is separate and must also be an exact permutation, never a subset. Legacy schema-1 contracts must be reviewed and reinitialized; the software does not silently reinterpret them.

The Future Code gate invokes the installed module or a pinned absolute bridge path outside the candidate:

```
python -m ichact.future_bridge \
  --contract /work/contracts/approved.json \
  --reports /work/verification-reports \
  --layout /work/contracts/layout.json
```

The working directory is the candidate snapshot. Exit 0 means complete current pass; nonzero means rejection or unknown. This CLI does not itself deploy or merge. The original harness performs its own staged-source fingerprint and integration checks. For hostile code, use an outer container/VM and do not give it access to secrets or trusted contract storage.

The artifact contains an actual scripted harness campaign, a separate live-endpoint configuration guide, deterministic tests, an independent audit, and an isolated reproduction launcher. The live guide is a deployment recipe, not evidence that paid inference or hosted evaluation was performed.